

Proyecto: Predicción de incumplimiento de pago

Fecha de entrega: 27 de junio del 2026

César Emiliano Garduño Gutiérrez
Daniel Arturo Castillo Rodríguez
Eduardo Martínez Vega
Erika Isabel Castillo Santiago
María Fernanda Zacateco Tello
Maximiliano Gómez Mendoza

Instrucciones: Elaborar un reporte técnico, con base en el código del proyecto, donde se explique el problema, la limpieza de datos, la ingeniería de variables, los modelos utilizados, la evaluación y las conclusiones principales.

Reporte técnico

1. Planteamiento del problema

El objetivo del proyecto es construir modelos de aprendizaje supervisado para predecir el incumplimiento de pago de clientes de tarjetas de crédito. La variable objetivo es `default payment next month`, donde el valor 1 representa incumplimiento y el valor 0 representa no incumplimiento.

Formalmente, para cada cliente i se busca estimar una probabilidad de default o incu:

$$\hat{p}_i = P(Y_i = 1 \mid X_i),$$

donde X_i contiene variables demográficas, historial de atraso, montos facturados, pagos realizados e indicadores creados durante la etapa de ingeniería de variables.

Este problema es relevante porque, en riesgo crediticio, no basta con clasificar correctamente a la mayoría de los clientes. La clase de mayor interés es la de impago, ya que no detectar a un cliente riesgoso puede representar pérdidas financieras para la institución.

2. Descripción de la base de datos

La base utilizada corresponde a clientes de tarjetas de crédito. Cada fila representa una cuenta y cada columna describe información financiera o demográfica del cliente. Las principales familias de variables son:

- **Límite de crédito:** `LIMIT_BAL`.
- **Variables demográficas:** `SEX`, `EDUCATION`, `MARRIAGE` y `AGE`.
- **Historial de atraso:** `PAY_0`, `PAY_2`, ..., `PAY_6`.
- **Montos facturados:** `BILL_AMT1`, ..., `BILL_AMT6`.
- **Pagos realizados:** `PAY_AMT1`, ..., `PAY_AMT6`.
- **Variable objetivo:** `default payment next month`.

En el análisis exploratorio se identificó que aproximadamente el 77.88 % de las observaciones pertenecen a la clase de no incumplimiento, mientras que alrededor del 22.12 % pertenecen a la clase de incumplimiento. Por lo tanto, la base presenta desbalance de clases.

Clase	Interpretación	Porcentaje aproximado
0	No incumplimiento	77.88 %
1	Incumplimiento	22.12 %

Cuadro 1: Distribución aproximada de la variable objetivo.

El desbalance de clases implica que la exactitud global, por sí sola, puede ser engañosa. Un modelo que prediga siempre la clase 0 podría obtener una exactitud alta, pero no sería útil para detectar clientes con riesgo de incumplimiento.

3. Limpieza y validación de datos

La primera etapa consistió en transformar la base original a un formato adecuado para el modelado. El archivo contenía los nombres de las columnas en la primera fila, por lo que fue necesario renombrar columnas y definir ID como índice.

```

1 base_sucia = pd.read_excel("default of credit card clients.xls")
2 base = base_sucia.rename(columns=base_sucia.iloc[0]).drop(base_sucia.
   index[0])
3 base = base.set_index("ID")

```

Listing 1: Carga y estructura inicial de la base

Después se revisaron valores nulos, tipos de datos y número de valores únicos por columna. Como los datos venían originalmente en formato `object`, se convirtieron de forma explícita a formatos numéricos.

Durante la validación se encontraron categorías atípicas en `EDUCATION` y `MARRIAGE`. Los valores 0, 5 y 6 de educación, así como el valor 0 en estado civil, se reagruparon dentro de la categoría de “otros”.

```

1 base["EDUCATION"] = base["EDUCATION"].replace({
2     0: 4,
3     5: 4,
4     6: 4
5 })
6
7 base["MARRIAGE"] = base["MARRIAGE"].replace({
8     0: 3
9 })

```

Listing 2: Corrección de categorías atípicas

También se revisó que variables como límite de crédito, edad y pagos realizados no tuvieran valores negativos. Finalmente, se identificaron 35 filas duplicadas, considerando todas las columnas excepto el identificador. Bajo el supuesto de que era poco probable que dos clientes tuvieran exactamente el mismo perfil financiero y demográfico, dichas observaciones fueron eliminadas.

4. Ingeniería de variables

Además de utilizar las variables originales, se construyeron variables auxiliares para resumir mejor el comportamiento crediticio de cada cliente. Estas variables capturan tres dimensiones principales: atraso, utilización del crédito y comportamiento de pago.

Variable	Descripción
avg_delay	Promedio de meses de atraso, considerando valores menores o iguales a cero como ausencia de atraso.
max_delay	Máximo atraso observado en el historial del cliente.
months_delayed	Número de meses en los que el cliente presentó atraso.
recent_delay	Indicador binario de atraso reciente, construido a partir de PAY_0.
avg_utilization	Utilización promedio del crédito, calculada como saldos facturados entre límite de crédito.
max_utilization	Máxima utilización del crédito observada.
total_payment_ratio	Razón entre pagos totales realizados y saldo total facturado.
bill_trend	Diferencia entre el saldo facturado más reciente y el más antiguo.
delay_trend	Cambio en el atraso entre el periodo más reciente y el más antiguo.
recent_delay_x_utilization	Interacción entre atraso reciente y utilización promedio.

Cuadro 2: Variables auxiliares creadas para el modelo.

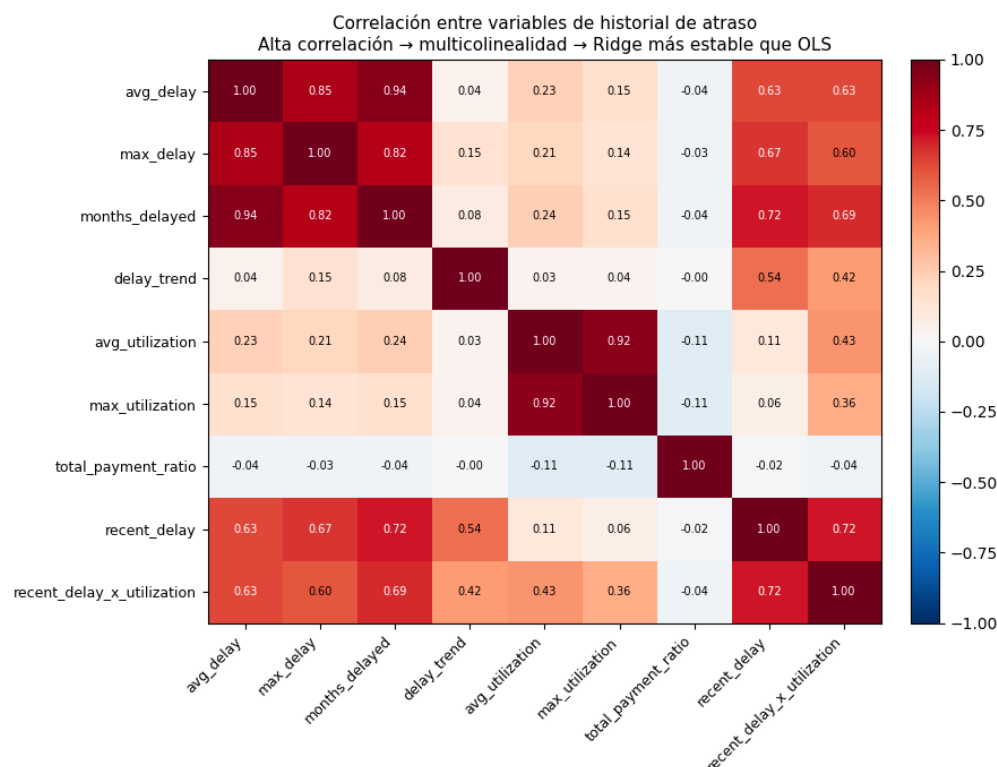


Figura 1: Correlación entre variables de historial de atraso y utilización del crédito.

Estas variables permiten que los modelos identifiquen patrones que no están resumidos directamente en las columnas originales. Por ejemplo, un cliente con varios meses de atraso, alto

atraso máximo y atraso reciente debería tener mayor probabilidad estimada de incumplimiento.

5. Preprocesamiento y diseño experimental

Después de la limpieza, la base se dividió en variables explicativas X y variable objetivo y . Posteriormente se realizó una partición de entrenamiento y prueba con proporción 80 %-20 %. Se utilizó una semilla aleatoria igual a 42 para asegurar reproducibilidad.

```

1 X = base.drop('default payment next month', axis=1)
2 y = base['default payment next month']
3
4 X_train, X_test, y_train, y_test = train_test_split(
5     X, y,
6     test_size=.2,
7     random_state=seed,
8     stratify=y
9 )

```

Listing 3: Separación de entrenamiento y prueba

El argumento `stratify=y` es importante porque conserva la proporción de clases en entrenamiento y prueba. Esto evita que, por azar, alguno de los conjuntos tenga una distribución diferente de impago y no impago.

Para la validación de modelos se utilizó `StratifiedKFold` con 5 particiones, lo cual mantiene la proporción de clases en cada pliegue de validación cruzada.

6. Métricas de evaluación

Como la base está desbalanceada, no se debe elegir el modelo solamente con accuracy. En este proyecto se utilizaron métricas que permiten evaluar mejor el comportamiento frente a la clase de impago.

Métrica	Interpretación
AUC-ROC	Mide la capacidad general del modelo para separar clientes con y sin incumplimiento.
Recall de impago	Proporción de clientes que incumplen y que el modelo detecta correctamente.
Precisión de impago	Proporción de clientes predichos como riesgosos que efectivamente incumplen.
F1 de impago	Promedio armónico entre precisión y recall para la clase de impago.
Balanced accuracy	Promedio del desempeño en ambas clases; es más informativa que accuracy cuando hay desbalance.

Cuadro 3: Métricas utilizadas para evaluar los modelos.

En este contexto, el recall de impago es especialmente importante porque mide la capacidad del modelo para identificar clientes riesgosos. Sin embargo, si se aumenta demasiado el recall, también pueden crecer los falsos positivos. Por ello, el F1-score y el threshold de decisión ayudan a encontrar un balance más razonable.

7. Modelos lineales

Primero se entrenaron modelos lineales como referencia. Estos modelos son útiles porque son interpretables y permiten entender qué variables aumentan o disminuyen la probabilidad de incumplimiento.

- 7.1. **Regresión logística sin regularización.** Sirve como modelo base. Se utilizó `StandardScaler` porque las variables tienen escalas diferentes. Además, se aplicó `class_weight = balanced` para compensar el desbalance de clases.
- 7.2. **Regresión logística con penalización L1, Lasso.** Esta penalización puede llevar algunos coeficientes exactamente a cero, por lo que funciona como mecanismo de selección de variables.
- 7.3. **Regresión logística con penalización L2, Ridge.** Esta penalización no elimina variables, sino que reduce la magnitud de los coeficientes. Es útil cuando hay multicolinealidad entre variables.

La regresión logística sin regularización obtuvo un AUC-ROC promedio de 0.7660 ± 0.0112 en validación cruzada y un AUC-ROC de 0.7561 en el conjunto de prueba. Esto indica que el modelo discrimina mejor que un clasificador aleatorio y que no presenta una diferencia preocupante entre entrenamiento y prueba.

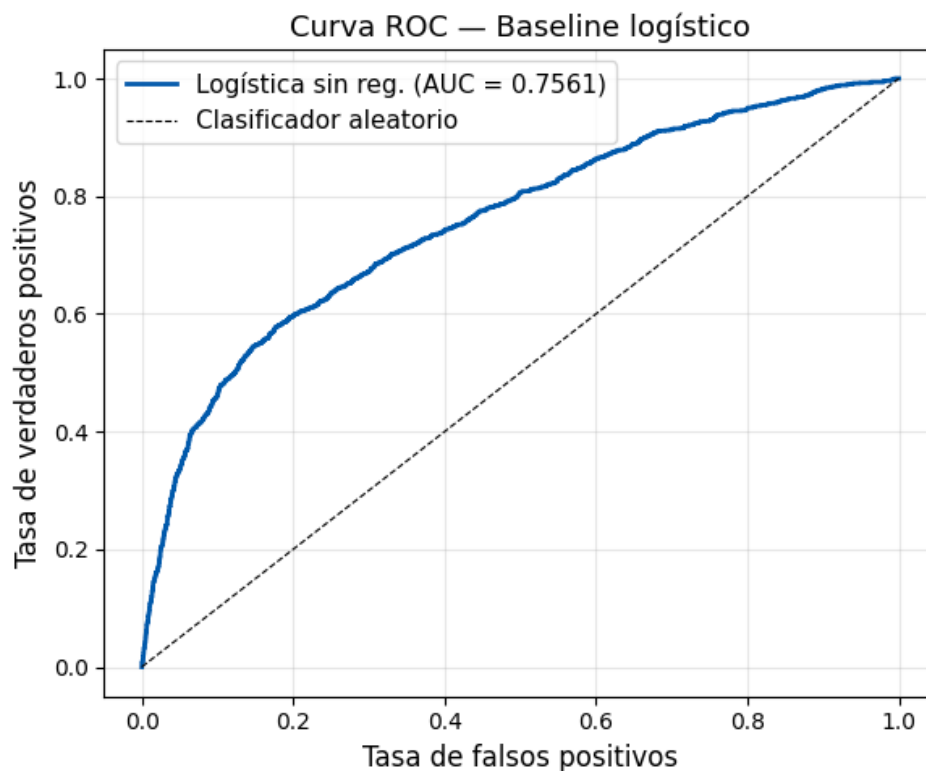


Figura 2: Curva ROC del modelo logístico base.

Lasso obtuvo un desempeño muy similar al modelo base. Con $C = 1,0$ no eliminó variables por completo, aunque el análisis de sparsity mostró que valores menores de C sí incrementan

el número de coeficientes iguales a cero. Ridge obtuvo también un desempeño parecido, con AUC-ROC promedio de 0.7660 ± 0.0111 . En la búsqueda de hiperparámetros, el mejor valor reportado para Ridge fue $C = 0.1$, con AUC-ROC aproximado de 0.7661.

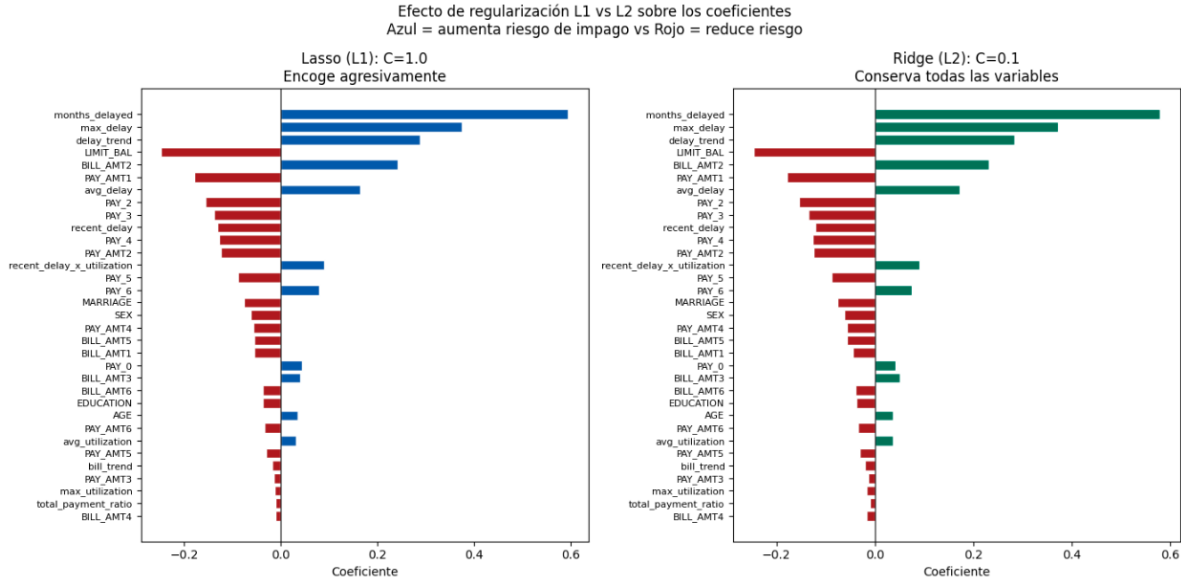


Figura 3: Comparación de coeficientes estimados mediante regularización L1 y L2.

8. Comparación entre usar y no usar ponderación de clases

Debido al desbalance entre clientes con y sin incumplimiento, se comparó una regresión logística Ridge sin ajuste de pesos contra una regresión logística Ridge con `class_weight = balanced`.

Modelo	AUC-ROC	Balanced Acc.	Recall impago	F1 impago
Sin ponderación	0.7540	0.6411	0.33	0.4404
Con ponderación	0.7561	0.6968	0.60	0.5151

Cuadro 4: Efecto de ponderar la clase minoritaria.

El modelo sin ponderación obtiene buen desempeño global, pero detecta pocos casos de impago. En cambio, al usar `class_weight = balanced`, el recall de impago sube de 0.33 a 0.60 y el F1 de impago aumenta de 0.4404 a 0.5151. Esto muestra que el ajuste de pesos permite que el modelo preste más atención a la clase minoritaria.

En riesgo crediticio, este resultado es importante porque puede ser más costoso no detectar a un cliente riesgoso que clasificar incorrectamente a un cliente bueno como riesgoso.

9. Análisis del threshold de decisión

Los modelos de clasificación producen probabilidades. Para convertir esas probabilidades en clases se define un umbral de decisión. Aunque el umbral estándar es 0.5, en problemas de riesgo crediticio puede ser conveniente modificarlo según el costo de los errores.

Threshold	Resultado observado	Interpretación
0.3	Recall de impago 0.913	Prioriza no dejar pasar clientes riesgosos.
0.4	Mayor sensibilidad	Mantiene enfoque hacia recall.
0.5	Umbral estándar	Balance inicial entre clases.
0.6	F1 de impago 0.525	Mejor equilibrio observado según F1.
0.7	Precisión 0.587 y recall 0.430	Modelo más conservador.

Cuadro 5: Efecto cualitativo del threshold de decisión.

El threshold de 0.6 fue el que obtuvo el mejor F1-score reportado para la clase de impago. Sin embargo, si el objetivo fuera detectar la mayor cantidad posible de clientes riesgosos, podría preferirse un umbral menor como 0.3 o 0.4.

10. Random Forest

Random Forest es un modelo de ensamble basado en múltiples árboles de decisión. A diferencia de los modelos lineales, puede capturar relaciones no lineales e interacciones entre variables. Además, no requiere escalamiento porque los árboles no dependen de la escala de las variables.

El modelo baseline de Random Forest obtuvo un AUC de entrenamiento de 1.0000 y un AUC de validación de 0.7670 ± 0.0069 . La diferencia entre ambos valores indica sobreajuste, ya que el modelo aprende casi perfectamente el conjunto de entrenamiento, pero no generaliza con la misma intensidad.

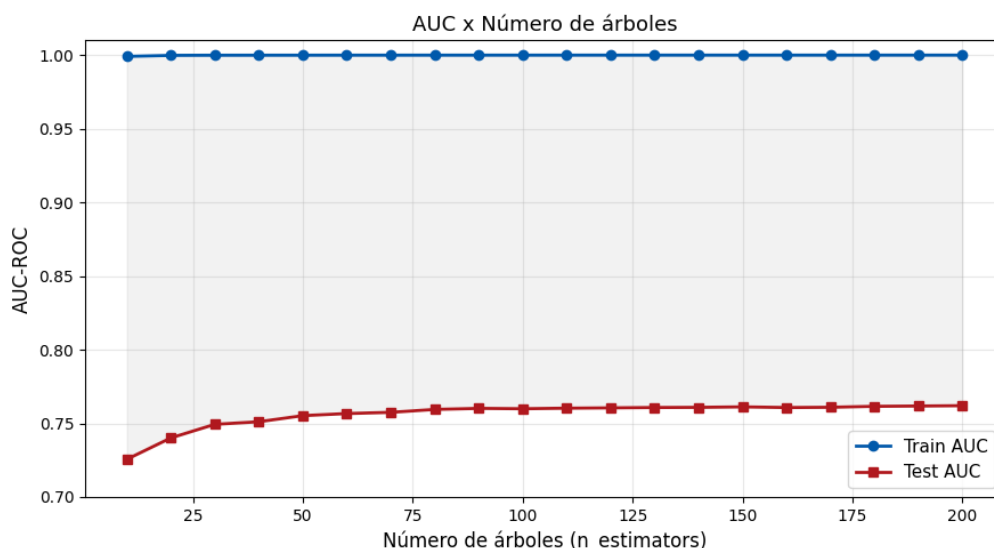


Figura 4: Evolución del AUC de entrenamiento y prueba según el número de árboles en Random Forest.

Por esta razón se aplicó `GridSearchCV` para ajustar hiperparámetros como:

- `n_estimators`, número de árboles.
- `max_depth`, profundidad máxima de cada árbol.
- `min_samples_leaf`, mínimo de observaciones por hoja.

También se calculó importancia de variables mediante importancia interna del bosque y mediante *permutation importance*. El objetivo fue identificar qué variables aportan mayor capacidad predictiva al modelo.

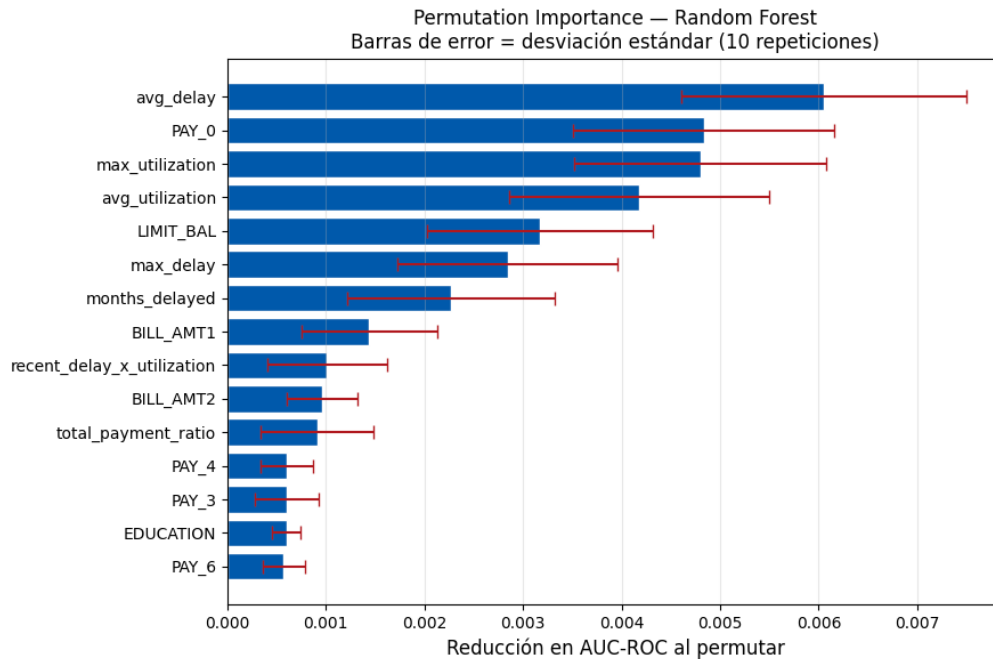


Figura 5: Importancia de variables por permutación en Random Forest.

11. AdaBoost

AdaBoost es un método de boosting que combina clasificadores débiles de manera secuencial. En este proyecto se usaron árboles de profundidad 1 como clasificadores base. La idea del algoritmo es que cada nuevo árbol se concentre más en las observaciones que los árboles anteriores clasificaron incorrectamente.

```

1 p_ada = Pipeline([
2     ('scaler', StandardScaler()),
3     ('model', AdaBoostClassifier(
4         estimator=DecisionTreeClassifier(max_depth=1),
5         n_estimators=100,
6         learning_rate=1.0,
7         random_state=seed,
8         algorithm='SAMME'
9     ))
10 ])

```

Listing 4: Estructura general de AdaBoost

Para AdaBoost se evaluaron diferentes valores de `n_estimators` y `learning_rate`. El código calcula AUC-ROC, balanced accuracy, recall, precision y F1 para la clase de impago. Aunque los valores exactos dependen de ejecutar el script completo con la base original, el propósito del modelo es comparar si un enfoque secuencial de boosting mejora la detección de impagos frente a los modelos lineales y Random Forest.

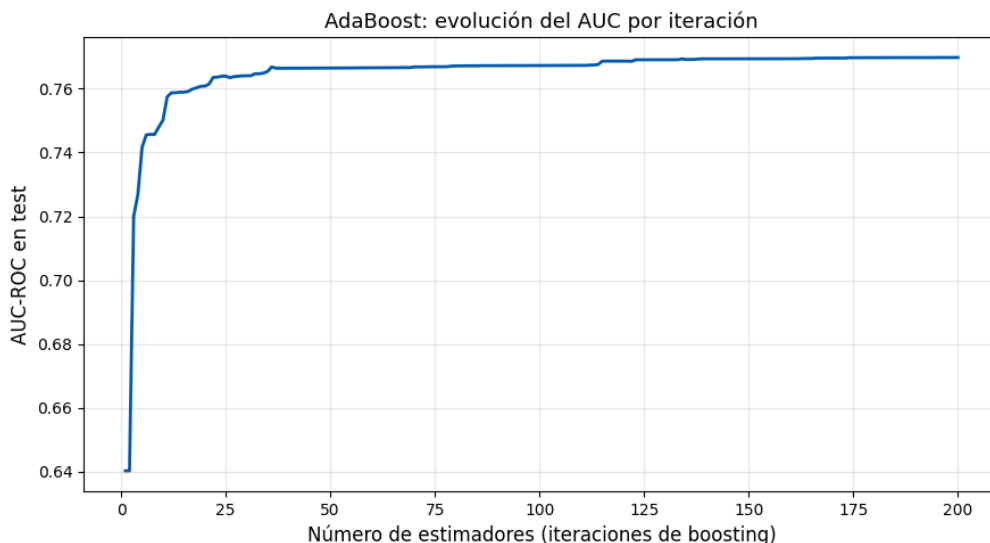


Figura 6: Evolución del AUC en prueba conforme aumentan las iteraciones de boosting en AdaBoost.

12. Comparación de modelos lineales, Random Forest y AdaBoost

Hasta esta etapa del proyecto, los modelos lineales mostraron un desempeño estable, con un AUC-ROC cercano a 0.766 en validación cruzada. La regresión logística funcionó como un modelo base sólido e interpretable, mientras que las regularizaciones L1 y L2 no mejoraron de forma importante la capacidad predictiva, pero sí aportaron valor para analizar la selección de variables y la estabilidad de los coeficientes.

El uso de `class weight = balanced` permitió mejorar la detección de clientes con incumplimiento, aumentando el recall y el F1-score de la clase minoritaria. Esto confirma la importancia de considerar el desbalance de clases y no evaluar los modelos únicamente con accuracy o AUC-ROC.

Por otro lado, Random Forest mostró capacidad para capturar relaciones no lineales, aunque su desempeño en entrenamiento evidenció sobreajuste, por lo que requiere ajuste de hiperparámetros. AdaBoost complementó el análisis al incorporar un enfoque de aprendizaje secuencial basado en clasificadores débiles.

En conjunto, estos resultados muestran que la selección del modelo debe considerar no solo el desempeño estadístico, sino también el objetivo del problema: detectar clientes riesgosos o mantener un equilibrio entre falsos positivos y falsos negativos. Esta comparación servirá como base para las siguientes etapas del proyecto.

13. XGBoost con ajuste de pesos

En nuestro caso la función de pérdida es la log-loss (entropía cruzada binaria), que es la que normalmente se usa para clasificación binaria. Lo particular de XGBoost frente a un *Gradient Boosting* normal es que no solo usa el gradiente de esta función para decidir cómo construir cada árbol, sino también la segunda derivada (el hessiano), lo cual en la práctica le permite encontrar mejores divisiones más rápido.

Además, XGBoost trae varios mecanismos contra el sobreajuste:

- **learning_rate**: cada árbol se suma multiplicado por un factor pequeño, para aprender poco a poco.
- **subsample** y **colsample_bytree**: cada árbol usa una muestra aleatoria de filas y columnas.
- **reg_alpha** y **reg_lambda**: penalizaciones L1 y L2 sobre los pesos de las hojas.
- **gamma** y **min_child_weight**: exigen que un corte reduzca la pérdida lo suficiente y que cada hoja tenga suficientes observaciones, evitando divisiones que solo memorizan ruido.
- **scale_pos_weight** (el que más nos interesó): hace que los errores sobre la clase minoritaria pesen más.

Elegimos XGBoost, tras probar regresión logística, Random Forest y AdaBoost, por tres razones. Primero, la relación entre el atraso y el default es muy no lineal —la tasa de default pasa de 11.71 % sin meses de atraso a 70.30 % con atraso en los seis meses—, algo que los árboles captan de forma natural y a un modelo lineal le cuesta. Segundo, las variables de atraso (**PAY_0**, **avg_delay**, **max_delay**, **months_delayed**) están muy correlacionadas entre sí, lo que desestabiliza a un modelo lineal (por eso recurrimos a Ridge) pero no afecta a los árboles. Tercero, XGBoost permite atacar el desbalance directamente con **scale_pos_weight** en la función de pérdida, lo cual era clave dado que solo el 22 % de los clientes cae en default.

13.1. Modelo base con **scale_pos_weight**

El modelo base ya usaba **scale_pos_weight** para el desbalance, con hiperparámetros orientados a evitar el sobreajuste: **n_estimators=300**, **learning_rate=0.05**, **max_depth=3**, **min_child_weight=5**, **gamma=0.5**, **subsample=0.8**, **colsample_bytree=0.8**, **reg_lambda=10** y **reg_alpha=0.1**.

Aparte de las métricas de siempre reportamos el PR-AUC (*average precision*), que resume la curva de precision-recall. Nos pareció importante reportarlo porque, con clases tan desbalanceadas, el ROC-AUC puede dar una impresión más optimista de la que realmente tiene el modelo sobre la clase que nos interesa.

13.2. ¿Qué tanto ayuda **scale_pos_weight**?

Para aislar el efecto de **scale_pos_weight**, entrenamos un XGBoost idéntico pero sin ese ajuste (Modelo sin Ponderación de Clases). La diferencia es clara: sin pesos, el recall sobre la clase de default es de apenas 36.27 % (se escapan casi dos de cada tres morosos); con **scale_pos_weight** sube a 65.31 %, a costa de menor precisión (más falsas alarmas). El intercambio vale la pena, porque dejar pasar a un cliente que incumplirá suele salir más caro que una alerta de más sobre alguien que sí paga.

13.3. Tuning de hiperparámetros

Para intentar mejorar el modelo base hicimos una búsqueda aleatoria (**RandomizedSearchCV**) con validación cruzada estratificada de 5 folds, sobre número de árboles, tasa de aprendizaje, profundidad, poda (**gamma**, **min_child_weight**), submuestreo y regularización L1/L2. Cuidamos dos cosas: hacer la búsqueda solo en el train (para no contaminar el test) y optimizar directamente PR-AUC, más adecuado que ROC-AUC bajo desbalance. El mejor modelo (PR-AUC en CV $\approx 0,57$; **max_depth=5**, **learning_rate=0.03**) resultó casi idéntico al base ponderado en el test (ROC-AUC $\approx 0,78$, PR-AUC $\approx 0,57$): las diferencias son marginales y en direcciones distintas, sin mejora real. Concluimos que la configuración inicial ya era razonable y no compensa complicarse con el modelo tuneado.

13.4. Elegir el umbral de decisión

El umbral por defecto de 0.50 no tiene nada de especial, menos con clases desbalanceadas. Para elegirlo sin contaminar el test, sacamos probabilidades *out-of-fold* sobre el train con `cross_val_predict` (cada observación recibe la probabilidad de un modelo que no la vio), barrimos umbrales y nos quedamos con el que maximiza el F1; ese umbral (0.56 para el tuneado y 0.58 para el base ponderado) se aplicó una sola vez al test. Así nunca usamos el test para decidir el punto de corte y la comparación entre ambos modelos es justa.

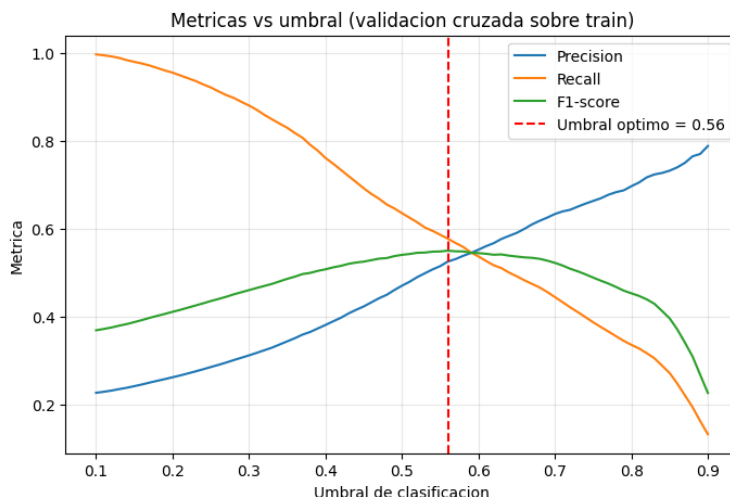


Figura 7: Barrido de umbrales con probabilidades *out-of-fold* sobre el conjunto de entrenamiento. El umbral que maximiza el F1 (línea punteada) se elige sin usar el conjunto de prueba.

13.5. Comparando todo

Juntamos las cinco variantes en una sola tabla: XGBoost sin pesos, XGBoost ponderado con umbral 0.50, XGBoost ponderado con su umbral óptimo, XGBoost tuneado con umbral 0.50 y XGBoost tuneado con su umbral óptimo. Ahí se puede ver de un vistazo el efecto de ponderar las clases, de hacer el tuning, y de mover el umbral, todo sobre las mismas métricas calculadas en el test. La Tabla 6 resume estos resultados.

Cuadro 6: Comparación de las cinco variantes de XGBoost sobre el conjunto de prueba. En negritas, el modelo seleccionado (ponderado, umbral 0.50), que ofrece el mayor recall.

Modelo	Umbral	Acc.	Prec.	Recall	F1	ROC-AUC	PR-AUC
Sin pesos	0.50	0.821	0.676	0.363	0.472	0.782	0.569
Ponderado	0.50	0.753	0.460	0.653	0.540	0.781	0.566
Ponderado	0.58	0.787	0.517	0.569	0.542	0.781	0.566
Tuneado	0.50	0.759	0.466	0.624	0.534	0.781	0.568
Tuneado	0.56	0.788	0.519	0.567	0.542	0.781	0.568

13.6. Qué variables importan más

Calculamos la importancia (ganancia) de cada variable en el modelo tuneado y graficamos las 15 principales (Figura 8). Coincide con el análisis exploratorio: dominan las varia-

bles de atraso que construimos (`max_delay`, `avg_delay`, `recent_delay`, `months_delayed`) junto con `PAY_0`, lo que confirma que el feature engineering aportó.

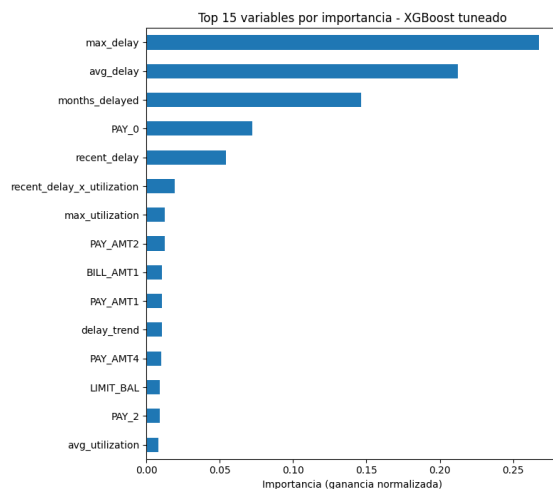


Figura 8: Importancia (ganancia) de las 15 variables más relevantes según el modelo XGBoost tuneado. Dominan las variables de atraso construidas (`max_delay`, `avg_delay`, `months_delayed`) y `PAY_0`.

13.7. Curvas precision-recall y ROC

Graficamos la curva ROC y la curva precision-recall del modelo tuneado sobre el test (Figura 9). Bajo desbalance, la curva precision-recall (y su área, el PR-AUC) refleja mejor el desempeño sobre la clase minoritaria que la ROC, que puede verse bien aunque al modelo le cueste detectar a los morosos. (Figura 9).

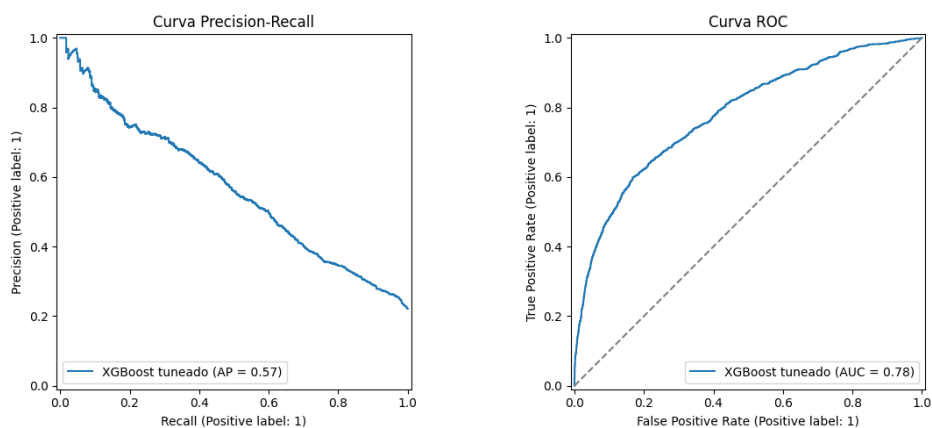


Figura 9: Curvas Precision-Recall (izquierda) y ROC (derecha) del modelo tuneado sobre el conjunto de prueba.

13.8. Calibrando las probabilidades

Al entrenar con `scale_pos_weight`, las probabilidades quedan sesgadas hacia la clase positiva y ya no se leen como "probabilidad real de impago": el modelo se optimizó para penalizar más esa clase, no para calibrar. Para medir ese sesgo compara-

mos la curva de calibración del modelo ponderado contra una versión recalibrada con `CalibratedClassifierCV` (método isotónico), que reordena las probabilidades hacia las frecuencias reales sin cambiar el orden de las predicciones (por eso ROC-AUC y PR-AUC no se afectan). Lo revisamos porque el proyecto habla en términos de probabilidad, no solo de una etiqueta sí/no. (Figura 10).

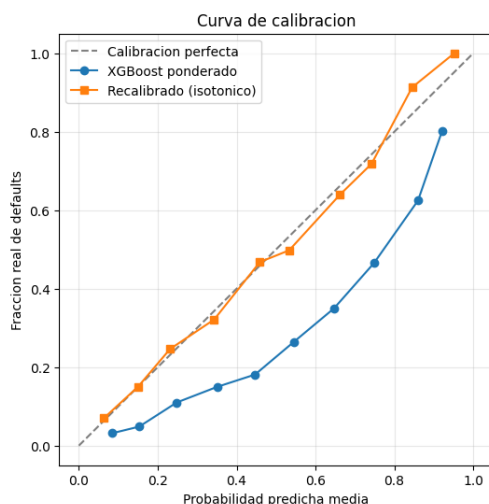


Figura 10: Curva de calibración: el modelo ponderado (sesgado por `scale_pos_weight`) frente a su versión recalibrada con el método isotónico. La diagonal representa la calibración perfecta.

13.9. El modelo que elegimos

Elegimos como modelo final el **XGBoost ponderado con umbral 0.50**. El modelo sin pesos logra mejor accuracy (0.82) y precision (0.67) pero un recall bajo (0.36); `scale_pos_weight` lo sube a 0.65 y el tuning no aportó mejora en el test. Como las cinco variantes tienen casi la misma discriminación ($\text{PR-AUC} \approx 0,57$, $\text{ROC-AUC} \approx 0,78$), la decisión depende del punto de operación, y en riesgo crediticio priorizamos el recall (el falso negativo es el error caro). El ponderado a 0.50 da el mayor recall: detecta 866 de 1,326 morosos (65.3 %) a cambio de 1,018 falsas alarmas sobre 4,667 buenos pagadores.

14. Red Neuronal

Introducción

Se implementó una Red Neuronal Artificial de tipo *Multilayer Perceptron* (MLP) para la clasificación del riesgo de impago. A diferencia de modelos lineales, un MLP puede aprender relaciones no lineales entre las variables de entrada mediante una o más capas ocultas.

Metodología para la selección de la arquitectura

Se realizó una comparación experimental entre distintas configuraciones del Perceptrón Multicapa (MLP) con el objetivo de identificar la arquitectura con mejor desempeño para la clasificación del riesgo de impago.

Los hiperparámetros evaluados fueron:

- Número de capas ocultas y neuronas por capa.
- Función de activación (ReLU y tangente hiperbólica).
- Parámetro de regularización L_2 (α).

Durante la evaluación se mantuvieron constantes el optimizador Adam, la tasa de aprendizaje, el tamaño del mini-batch, el criterio de *Early Stopping* y la semilla aleatoria para garantizar la reproducibilidad.

En total se evaluaron dieciséis configuraciones con una, dos y tres capas ocultas. Para cada una se calcularon las métricas ROC-AUC, *precision*, *recall*, puntaje F_1 y el número de épocas necesarias para la convergencia. Los resultados se resumen en la Tabla 7, ordenada de acuerdo con el valor de ROC-AUC.

Cuadro 7: Comparación de las configuraciones evaluadas para el MLP ordenadas por ROC-AUC.

Configuración	Activación	α	ROC-AUC
(64,)	relu	0.01	0.77
(64,128,32)	relu	0.01	0.7699
(64,32,128)	relu	0.01	0.7694
$\vdots$	$\vdots$	$\vdots$	$\vdots$

Discusión de la selección del modelo

La arquitectura con una capa oculta de 32 neuronas y función de activación *tanh* obtuvo el mayor valor de ROC-AUC. Sin embargo, la diferencia respecto a la arquitectura de 64 neuronas con ReLU fue mínima, por lo que la selección del modelo final también consideró el comportamiento observado en la matriz de confusión, en donde esta última obtuvo mejor *recall* (más verdaderos positivos). En este problema, un falso positivo implica aprobar a un cliente que posteriormente incumplirá sus obligaciones, mientras que un falso negativo corresponde a rechazar a un cliente confiable, por lo que, al considerar que los falsos positivos (no detectar un cliente con alta probabilidad de impago) cuesta más que detectar falsamente a un cliente como probable de impago, nos decantamos por la arquitectura de 64 neuronas con ReLU.

Se observó que la arquitectura de 64 neuronas con ReLU mejoraba la detección de clientes con riesgo de impago, obteniendo un mayor *recall* para la clase positiva. Dado que el objetivo principal del sistema es identificar oportunamente a estos clientes, se seleccionó como modelo final una arquitectura con una capa oculta de 64 neuronas, función de activación ReLU y regularización L_2 con $\alpha = 10^{-2}$.

Resultados obtenidos

El modelo seleccionado fue entrenado con el conjunto de entrenamiento y evaluado sobre el conjunto de prueba. Para analizar su desempeño se utilizaron la matriz de confusión, la curva ROC, la curva Precision–Recall y la evolución de la función de pérdida.

La Figura 11 muestra la curva ROC del modelo final, así como la evolución de la función de pérdida durante el entrenamiento. Se observa una convergencia de la evolución de pérdida, así como un aplanamiento en la curva ROC, llegando a un punto óptimo ambas métricas.

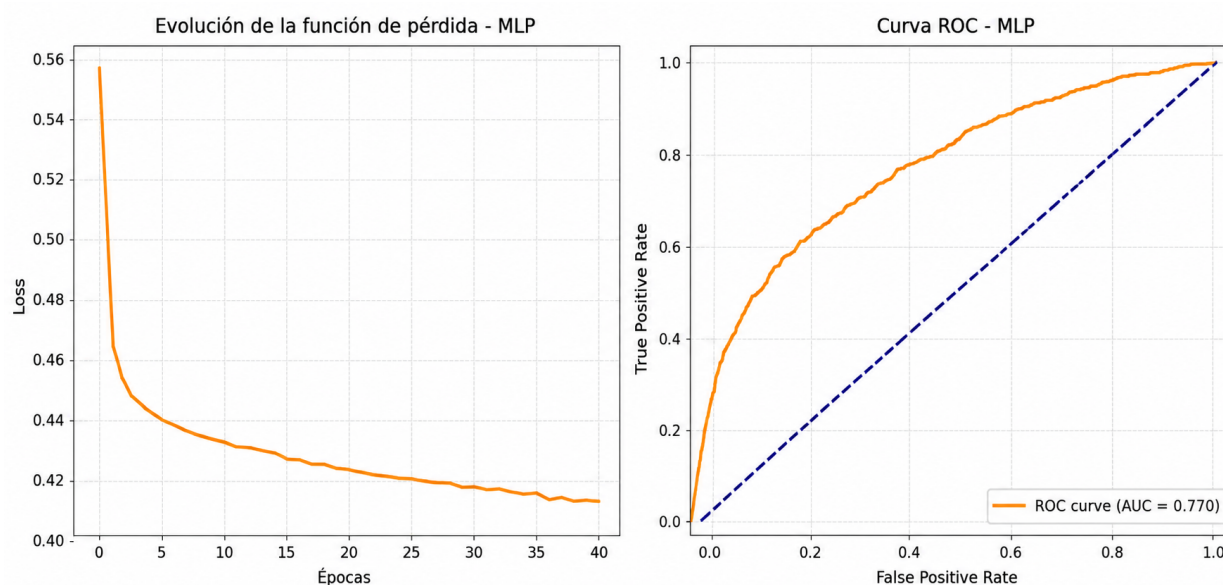


Figura 11: Evolución de pérdida (izq.) y Curva ROC (der.)

15. Comparación general de modelos

La tabla 8 resume los modelos con las métricas ya discutidas a lo largo del documento, en donde el modelo XGBoost resultó con el mayor AUC de todos los modelos.

Modelo	AUC CV	AUC Test	Observación principal
Logística sin regularización	0.7660 ± 0.0112	0.7561	Modelo base estable e interpretable.
Logística L1, Lasso	0.7660 ± 0.0112	—	Útil para estudiar selección de variables.
Logística L2, Ridge	0.7660 ± 0.0111	0.7561	Más estable ante multicolinealidad.
Random Forest baseline	0.7670 ± 0.0069	—	Captura no linealidad, pero mostró sobreajuste.
AdaBoost	Calculado en script	Calculado en script	Evalúa boosting secuencial con árboles débiles.
Red neuronal	$.7750 \pm 0.0184$	0.7700	El modelo neuronal logra capturar cierta no linealidad, superando marginalmente el desempeño de modelos de regresión logística
XGBoost ponderado	0.7884 ± 0.0096	0.7814	Mejor AUC y mayor recall (0.65); maneja el desbalance con <code>scale_pos_weight</code> .

Cuadro 8: Resumen comparativo de los modelos desarrollados.

16. Conclusiones

En este trabajo se evaluaron distintos modelos de aprendizaje supervisado para la predicción del riesgo de impago utilizando la base de datos *Default of Credit Card Clients*. El análisis permitió comparar enfoques lineales, métodos basados en árboles de decisión y redes neuronales bajo un mismo problema de clasificación desbalanceada.

Los resultados mostraron que todos los modelos fueron capaces de discriminar entre clientes con y sin riesgo de impago, obteniendo valores de ROC-AUC superiores a 0.75. La regresión logística destacó por su simplicidad, estabilidad e interpretabilidad, mientras que la regularización L_1 permitió identificar las variables más relevantes asociadas al incumplimiento crediticio.

Por otra parte, la Red Neuronal logró capturar relaciones no lineales presentes en los datos, obteniendo un desempeño superior al de la regresión logística, aunque la mejora fue moderada. Finalmente, XGBoost presentó el mejor rendimiento global, alcanzando un ROC-AUC de 0.7814 y una mayor capacidad para detectar clientes con riesgo de impago, gracias al uso de árboles potenciados y al ajuste del desbalance mediante *scale_pos_weight*.

En conjunto, los resultados evidencian que la elección del modelo depende no sólo de su desempeño predictivo, sino también de aspectos como la interpretabilidad, el costo computacional y el objetivo de la aplicación. Mientras que modelos como la regresión logística resultan adecuados cuando se requiere explicar las decisiones del modelo, algoritmos más complejos como XGBoost y redes neuronales ofrecen una mayor capacidad predictiva cuando el objetivo principal es maximizar la detección de clientes con riesgo de incumplimiento, aunque a costa de interpretabilidad.

En conclusión, la comparación realizada permitió comprender las fortalezas y limitaciones de cada técnica de aprendizaje supervisado y demostró que, para este conjunto de datos, XGBoost constituye la alternativa con mejor equilibrio entre capacidad predictiva y manejo del desbalance de clases en nuestra base de incumplimiento de pago de crédito.